

HW06 — API Testing Report (EShop)

- **Sinh viên:** Lê Nhựt Duy — **MSSV:** 23127178
- **Môn:** Kiểm thử phần mềm (QA/QC) — **Bài:** HW06-AI API Testing
- **SUT:** EShop — <https://github.com/ttbhanh/eshop-sut> · fork nhóm `DuyITL0R/group05_eshop` commit `f0f3b7b`
- **Công cụ:** Claude Code (Opus 5) · Postman collection + Newman `6.2.2` · Node `v22.23.1` · macOS `26.1 arm64`

Nguồn số liệu duy nhất: [test-cases/test-summary/summary.md](#) , sinh bằng `npm run summary` từ `reports/newman/*.json` . Không có con số nào trong báo cáo này được gõ tay.

Tóm tắt một trang

API kiểm thử	3 — mỗi pool một API, không trùng thành viên nhóm (docs/api-selection.md)
Test case	157 — 114 do AI sinh (lượt 1) + 22 do AI sinh (lượt 2, <code>AI-2</code>) + 21 do sinh viên chọn (<code>SV</code> , §6.3)
Đã thực thi	192 request · 372 assertion trên SUT thật ở <code>localhost:3000</code> · thêm regression suite 253 assertion, 0 đỏ
Kết quả	279 assertion xanh · 93 đỏ — mọi assertion đỏ đều map tới một bug
Bug	22 bug xác nhận (19 từ bộ AI + 3 từ case sinh viên chọn) + 2 rủi ro + 1 câu hỏi nghiệp vụ · Issues #323–#341 , #402–#407 · 5 Critical, 6 High, 9 Medium, 2 Low
Bug đáng chú ý nhất	BUG-14: khách không đăng nhập làm sập cả backend bằng 2 request — chuỗi 3 lỗi
Lỗi của AI đã bắt và sửa	26 (bảng đầy đủ ở §11) — 5 lỗi thiết kế test case (gồm 2 case assertion nghiêm hơn expected), 5 lỗi kỹ thuật, 4 lỗi bỏ sót phân vùng, 2 lỗi số liệu, 2 lỗi quy trình
Giả thuyết đã loại sau khi kiểm	4 (ghi lại ở bug-report §Bug đã loại)

Mục lục

1. [Phạm vi và lý do chọn API](#)
2. [Quy trình dùng AI từng bước](#)
3. [API-01 — GET /api/products](#)
4. [API-02 — POST /api/cart](#)
5. [API-03 — PUT /api/products/:id](#)
6. [Thực thi Postman + Newman](#)
7. [Bug](#)
8. [Postman feature đã dùng](#)
9. [CI/CD](#)
10. [AI test generator](#)
11. [Human review — AI sai và bỏ sót gì](#)

1. Phạm vi và lý do chọn API

Mã	Po ol	FR	API chính	Endpoint hỗ trợ trong cùng chuỗi test	Test case
API-01	A	FR-05 Product listing & search	GET /api/products?search=	GET /api/products/:id , POST /DELETE /api/products	48
API-02	B	FR-07 Shopping cart	POST /api/cart	GET /api/cart , POST /api/checkout , POST /api/login , DELETE /api/admin/users/:id	55
API-03	C	FR-15 Product management (admin)	PUT /api/products/:id	POST /GET /DELETE /api/products , POST /DELETE /api/categories	54

Bảng chứng không trùng nhóm (bảng 4 bộ API các thành viên khác đã đăng ký) và lý do chọn rút từ source: docs/api-selection.md .

Tiêu chí chọn được đặt trước khi đọc kết quả: mỗi API phải có ≥1 tham số phân hoạch được và ≥1 giả thuyết bug rút từ source, vì §6.1 đòi phân hoạch miền trên mọi tham số — một endpoint chỉ có một tham số :id không thể chứa 35 test case có nghĩa.

2. Quy trình dùng AI từng bước (§2, §6.1)

Đề §2 cấm đích danh một prompt gộp kiểu "generate all the API test cases from the spec and run them". Quy trình thực tế gồm 5 bước riêng biệt, mỗi bước một lượt AI và một mục trong ai-audit/ai-audit-report.md :

Bur ớc	Việc	Vào	Ra
1	Rút tham số · kiểu · ràng buộc · chỗ spec im lạng	api_specification.md + FR/SEC trong README SUT + backend/server.js	bảng tham số
2	Phân hoạch miền cho từng tham số (kể cả Authorization)	bảng tham số	case Domain
3	State transition — chuỗi request có thứ tự	bảng tham số	case State
4	Security SEC-01...SEC-07	README SUT	case Security
5	Schema validation — so với spec, không so với chính response	spec	case Schema

Sau đó là bước audit (§6.2) và bước tự thêm case (§6.3) — làm riêng, không cùng lượt với bước sinh.

Ba nguồn, không một nguồn. Mọi bước đọc cả spec API, cả FR/SEC, cả source. Lý do: expected phải bám spec, nhưng chỗ spec và code lệch nhau chính là bug — chỉ đọc một nguồn thì không thấy chỗ lệch. Cột Căn cứ của mỗi test case ghi rõ expected đến từ đâu (spec §3.1 , SEC-05 , FR-07 , server.js:161).

Một nguồn sự thật cho cả bảng lẫn collection. 157 test case được định nghĩa trong `generator/specs/<api>.mjs` ; `tools/gen-artifacts.mjs` sinh ra **cả** bảng Markdown 12 cột **và** collection Postman từ cùng định nghĩa đó. Viết tay hai chỗ thì bảng và collection lệch nhau ngay lần sửa đầu, và không ai phát hiện — người chấm đọc bảng, Newman chạy collection. Đây cũng là bản hiện thực của generator ở §10.

3. API-01 — Pool A • `GET /api/products`

3.0 Đặc tả và tham số

spec §3.1: `GET /api/products` , query `?search=keyword` (*tuỳ chọn*) — tìm sản phẩm **theo tên**. Endpoint public (mục §3 không đòi `Authorization`). Endpoint verify: §3.2 `GET /api/products/:id` .

Tham số	Nơi	Kiểu	Bắt buộc	Ràng buộc từ spec	Spec im lặng về
<code>search</code>	query	string	không	tìm theo <code>name</code>	trim · độ dài tối đa · phân biệt hoa/thường · ký tự đặc biệt
<code>:id</code> (verify)	path	integer	có	khoá tự tăng	status khi không tồn tại

Chỗ spec im lặng được xử lý bằng một trong hai cách, **không bao giờ bịa expected**: (a) suy từ FR-05/SEC và ghi rõ suy từ đâu, hoặc (b) chỉ khẳng định phần spec bảo đảm (status + schema). Ví dụ TC-PRODLIST-011 (`search=" "`) chỉ kiểm 200 + schema — xem §11 lỗi #1.

3.1–3.4 Phân bố test case

Kỹ thuật	Số case	Ví dụ
Domain partition	18	không truyền · rỗng · hoa/thường · tiếng Việt có dấu · 1 ký tự · 300 ký tự · emoji · <code>%</code> · <code>_</code> · <code>'</code> · tham số lạ · method sai
State transition	5	tạo → tìm thấy → xoá → không tìm thấy → chi tiết sản phẩm đã xoá
Security	6	SEC-05: tautology · comment · UNION · stacked query · boolean-based; SEC-04: payload XSS
Schema validation	14	kiểu từng field · không lộ field lạ · id lẻ/chẵn · id không tồn tại/sai kiểu/âm/0 · response lỗi

Bảng đầy đủ 12 cột: `generated.md` · `audit.md` · `extended.md` · `own.md` — 5 case do sinh viên chọn (§6.3).

3.5 Audit (§6.2)

36 case AI sinh: **35 VALID** · **1 INCOMPLETE (đã sửa)**. Case sửa là TC-PRODLIST-011 — chi tiết §11 lỗi #1. Nguyên tắc giữ suốt bài: **không sửa expected cho khớp hành vi sai của SUT**. 14 case đỏ ở API-01 là phát hiện, không phải lỗi test.

3.6 Case tự thêm (§6.3) — 7 case

`TC-PRODLIST-101...107` . Ba case đáng chú ý:

- **101** `search=áo` (chữ thường có dấu) → **BUG-05**. AI chỉ sinh biến thể `Áo`. Lý do bỏ sót: *characteristics of the API* — `LIKE` của SQLite chỉ không phân biệt hoa/thường với **ASCII**.
- **102/103** `%` là **ký tự hợp lệ trong tên sản phẩm** ("bàn phím 100%"), không chỉ là payload tấn công → **BUG-06**. Lý do: *model limitations* — AI gắn `%` với ngữ cảnh SQLi nên bỏ mất phân vùng dữ liệu hợp lệ.
- **106** kiểm **hệ quả** của `'; DROP TABLE products--`: bảng còn nguyên hay không. Lý do: *model limitations* — AI đánh giá test security qua status code.

Bảng "vì sao AI bỏ sót" đầy đủ 7 dòng ở `extended.md`.

3.7 Kết quả và bug

62 request · 155 assertion · 126 xanh · 29 đỏ. Bug: **BUG-01** (SQLi, Critical), **BUG-02** (lỗi DB trả HTML + lộ `SQLITE_ERROR`, High), **BUG-03** (không tồn tại → `200 {}`), **BUG-04** (`price` đổi kiểu theo chẵn/lẻ của `id`), **BUG-05** (tiếng Việt), **BUG-06** (wildcard `LIKE`).

4. API-02 — Pool B · `POST /api/cart`

4.0 Đặc tả và tham số

spec §4.2: body `{id, name, price, quantity}`, header `Authorization: Bearer <token>`.

Tham số	Kiểu	Ràng buộc từ spec	Suy thêm từ FR
<code>id</code>	integer	<code>id: 1</code>	phải là sản phẩm đang tồn tại (FR-07)
<code>name</code>	string	—	phải khớp sản phẩm của <code>id</code>
<code>price</code>	number	<code>price: 100000</code>	phải bằng giá catalog (FR-07 + FR-08) — xem ghi chú dưới
<code>quantity</code>	integer	<code>quantity: 2</code>	≥ 1 (FR-07)
<code>Authorization</code>	header	Bearer JWT	SEC-02

Ghi chú trung thực về `price`. spec §4.2 **có** ghi `price` trong body, nên đọc thuần câu chữ thì gửi giá là *đúng đặc tả*. Bài này vẫn khẳng định giá trong giỏ phải bằng giá catalog, vì FR-08 tính tiền đơn hàng từ giỏ: nếu client đặt giá thì client đặt luôn số tiền phải trả. Đây là **suy luận từ FR-07/FR-08**, được ghi đúng như vậy ở cột **Căn cứ** và trong `audit.md` để người chấm tự đánh giá lập luận.

4.1–4.4 Phân bố

Kỹ thuật	Số case	Nội dung
Domain	23	<code>quantity</code> (9 phân vùng) · <code>id</code> (6) · <code>price</code> (5) · <code>name</code> (3)
State	7	rỗng → thêm → đọc giỏ → giá đúng catalog → thêm lại → một sản phẩm một dòng → checkout → giỏ rỗng → cách ly giỏ giữa 2 user
Security	6	SEC-02 (4 biến thể header) · SEC-05 payload trong <code>name</code> · SEC-06 mass assignment

Kỹ thuật	Số case	Nội dung
Schema	3	{message} · schema giỏ (quantity ≥ 1) · 401 {error}

4.5 Audit

39 case AI sinh: **38 VALID · 1 INCOMPLETE (đã sửa)** — TC-CART-008 (quantity=999999 "vượt tồn kho"): bảng products của SUT **không có** cột tồn kho (database.js:64-72), nên ràng buộc FR-07 này không có dữ liệu để kiểm. Giữ case + ghi rõ hạn chế, thay vì xoá case cho bảng đẹp. Xem §11 lỗi #2.

4.6 Case tự thêm — 7 case

TC-CART-101...107 . Điểm chung của cả 7: đều là câu hỏi "hệ quả là gì" thay vì "status code là gì".

- **101/102** price tampering + đọc lại giỏ để chứng minh giá 1 đồng đã vào giỏ → **BUG-08** (Critical). Lý do bỏ sót: *prompt quality* — phân hoạch một tham số **độc lập** không bao giờ tìm ra lỗi kiểu "giá trị hợp lệ nhưng sai so với dữ liệu khác".
- **103** checkout **hai lần liên tiếp** → **BUG-09**. Lý do: *characteristics of the API* — checkout chỉ INSERT vào orders , không xoá giỏ (server.js:297-309).
- **104** field lạ có **được lưu** không → **BUG-10**. **107** bất biến trạng thái giỏ sau khi bị bơm input rác → tổng hợp **BUG-07**.
- **105/106** xoá sản phẩm khỏi catalog rồi thêm vào giỏ → **BUG-11**. Lý do: *prompt quality* — prompt gán "state transition" cho **đơn hàng** (FR-10), không ai nói rằng **catalog** cũng có trạng thái.

4.7 Kết quả và bug

52 request · 85 assertion · 55 xanh · 30 đỏ. Bug: **BUG-07** (không validate gì, High), **BUG-08** (price tampering, Critical), **BUG-09** (giỏ không xoá sau checkout, High), **BUG-10** (mass assignment), **BUG-11** (sản phẩm không tồn tại), **BUG-12** (trùng dòng).

Hai giả thuyết bị loại sau khi thử thật: cách ly giỏ giữa hai user **hoạt động đúng** (TC-CART-030 xanh) và SEC-02 **đạt** ở endpoint này (TC-CART-031...034 xanh). Ghi lại để không nhận vơ.

5. API-03 — Pool C · PUT /api/products/:id

5.0 Đặc tả và tham số

spec §3.3 "*Thêm / Sửa / Xóa Sản phẩm (Dành cho Admin)*", body 5 field {name, price, description, imageUrl, category_id} . Ràng buộc quyền: **SEC-02 + SEC-03**.

Tham số	Kiểu	Ràng buộc	Nguồn
:id	integer	sản phẩm phải tồn tại	spec §3.3 (Cập nhật)
name	string	không rỗng	FR-15
price	number	> 0	đề §6.1 nêu đích danh ví dụ price > 0
category_id	integer	phải tồn tại trong categories	spec §3.4 · FR-14

Tham số	Kiểu	Ràng buộc	Nguồn
imageUrl	string	spec im lạng về validate URL	spec §3.3
Authorization	header	JWT và role='admin'	SEC-02, SEC-03

5.1–5.4 Phân bố

Kỹ thuật	Số case	Nội dung
Domain	20	name (4) · price (6) · category_id / imageUrl (5) · :id (5)
State	6	đặt nền → verify → xoá → cập nhật sản phẩm đã xoá → không hồi sinh trong danh sách
Security	9	SEC-02 (4 biến thể header) · SEC-03 role escalation · SEC-05 payload ở :id + verify tác động · SEC-06 mass assignment + verify
Schema	6	{message} · schema product sau update · không lộ field lạ · giá lớn không bị làm tròn

5.5 Audit

39 case AI sinh: **38 VALID · 1 INCOMPLETE (đã sửa)** — TC-PRODUPD-005 (name 300 ký tự): AI ghi cứng 400 trong khi spec không nêu giới hạn độ dài. Đã hạ về "không được 500 + response vẫn là JSON". Xem §11 lỗi #3.

Một quyết định thiết kế được ghi lại trong audit: chuỗi làm **sập SUT** (BUG-14) **không** nằm trong collection. 00-setup chọn id lẻ làm fixture và mọi request verify chỉ đọc id lẻ, vì GET /api/products/:id với id **chẵn** + price = NULL sẽ giết tiến trình backend và làm mọi case phía sau đổ **vì môi trường**. Chuỗi đó được tái hiện riêng bằng bug-report/verify-bugs.sh (có khởi động lại SUT).

5.6 Case tự thêm — 8 case

TC-PRODUPD-101...108 . Hai nhóm:

- **101/102/103** — không dừng ở status code mà GET lại để chứng minh **dữ liệu đã đổi thật** khi PUT không token và khi PUT bằng token user thường → **BUG-13** lên mức Critical. Lý do bỏ sót: *model limitations* và *prompt quality*.
- **104/105** partial update → đọc lại thấy price / description / category_id thành null → **BUG-15**, đồng thời là **mất thứ hai của BUG-14**. Lý do: *characteristics of the API* — phải đọc câu UPDATE mới đặt ra câu hỏi.
- **106** PUT vào :id không tồn tại có **tạo hàng mới** (upsert) không → xác nhận chỉ báo sai, không tạo.
- **107/108** soát **route lân cận cùng nhóm quyền**: POST và DELETE /api/products cũng thiếu authenticateToken . Lý do: *prompt quality* — prompt khoanh vùng "the API you selected", nhưng thiếu middleware là lỗi ở **mức router**; một báo cáo chỉ nói về PUT sẽ khiến người sửa vá một dòng và để nguyên hai lỗi còn lại.

5.7 Kết quả và bug

57 request · 93 assertion · 63 xanh · 30 đỏ. Bug: **BUG-13** (không có auth, Critical), **BUG-14** (DoS, Critical), **BUG-15** (NULL đề dữ liệu, High), **BUG-16** (không validate, High), **BUG-17** (id không tồn tại vẫn 200), **BUG-18** (mất chính xác tiền).

6. Thực thi Postman + Newman (§6.4)

```
npm run preflight      # SUT sống? tài khoản seed còn? 3 API phản hồi?
npm run test:all       # 3 collection, mỗi collection chạy trên SUT vừa khởi động lại
npm run summary        # reports/newman/*.json → test-cases/test-summary/summary.md
```

Header X-Student-Id (§6.4, §11). Bằng chứng: [postman-console-gui.png](#) — **Postman Console trong GUI**, 12 dòng do pre-request script in ra, mỗi dòng kèm request tới `http://localhost:3000` trả `200`; [console-x-student-id.png](#) (khối CONSOLE LOGS trong báo cáo Newman) và [x-student-id-request-header.png](#) (bảng REQUEST HEADERS). Đặt bằng **pre-request script cấp collection** ([postman/prerequest-collection.js](#)) chứ không gắn tay từng request: 171 request thì gắn tay là 171 chỗ có thể sót, và sót một chỗ là mất bằng chứng §11 cho request đó. Script đồng thời `console.log` để chụp được trong Postman Console; output Newman cũng in ra từng dòng `[HW06] X-Student-Id = 23127178 | GET /api/products | <timestamp>` — thấy trong [reports/newman/*.html](#).

Hostname (§11): mọi lượt chạy trên `http://localhost:3000`, đúng nơi triển khai của sinh viên.

Vì sao runner tự khởi động lại SUT trước mỗi collection. `backend/database.js:15–20 DROP rồi seed lại toàn bộ bảng` mỗi lần start. Đó vừa là ràng buộc vừa là công cụ: khởi động lại là cách duy nhất có **trạng thái đầu vào xác định**. Chạy lượt thứ hai trên CSDL đã bị 157 test case sửa dữ liệu hai lượt không so được với nhau. Runner chỉ kill tiến trình **do chính nó khởi động** (PID trong `.run-logs/sut.pid`), không `kill` theo tên — máy có thể đang chạy backend của bài khác.

Tính tái lập — đã kiểm bằng 3 lượt liên tiếp:

Lượt	API-01	API-02	API-03
1 local	29/155	30/81	30/93
2 local	29/155	30/81	30/93
3 local	29/155	30/81	30/93
4 CI (runner Ubuntu)	29/155	30/85*	30/93
5 CI (lượt XANH §9)	29/155	30/85*	30/93
6–7 local (sau khi sửa TC-020/021)	29/155	30/85	30/93
8 local (bộ nộp — sinh viên tự chạy 23/08 00:09)	29/155	30/85	30/93

* API-02 có 85 assertion thay vì 81 sau khi sửa TC-CART-020/021 (§11 lỗi #16) — số **đỏ** không đổi.

Số đỏ giống nhau tuyệt đối qua **8 lượt trên 2 hệ điều hành**, trong đó lượt cuối do **sinh viên tự chạy**. Điều này **không** có sẵn: lượt kiểm tái lập đầu tiên cho API-02 ra **34** thay vì 30, và truy nguyên ra một lỗi môi

trường sâu hơn lỗi #10 — xem §11 lỗi #12. Chỉ giữ file của lượt mới nhất trong `reports/newman/` để bộ nộp không phình; hai lượt trước đã xóa sau khi ghi lại số liệu ở bảng này.

Kết quả (sinh tự động):

API	Request	Assertion	Passed	Failed
API-01 · GET /api/products	67	170	141	29
API-02 · POST /api/cart	61	99	68	31
API-03 · PUT /api/products/:id	64	103	70	33
Tổng	192	372	279	93
(regression suite — cổng CI)	120	253	253	0

7. Bug (§6.5)

22 bug xác nhận + 2 rủi ro + 1 câu hỏi nghiệp vụ, tất cả tái hiện được bằng request thật, 25/25 đã mở GitHub Issue — [#323–#341](#) trên `DuyITL0R/group05_eshop`, mỗi issue đủ 8 trường của template và có ảnh báo cáo Newman nhúng sẵn. Bản đầy đủ: [bug-report/bug-report.md](#) · log tái hiện: [bug-report/verify-bugs-output.txt](#).

Severity	Bug
Critical	BUG-01 SQLi · BUG-08 price tampering · BUG-13 thiếu hoàn toàn auth ở 3 route · BUG-14 DoS làm sập backend · BUG-19 mật khẩu plaintext (SEC-01)
High	BUG-02 lỗi DB trả HTML + lộ <code>SQLITE_ERROR</code> · BUG-07 không validate giỏ · BUG-09 giỏ không xóa sau checkout · BUG-15 NULL đề dữ liệu · BUG-16 không validate sản phẩm
Medium	BUG-03 · BUG-04 · BUG-05 · BUG-06 · BUG-10 · BUG-11 · BUG-17
Low	BUG-12 · BUG-18

BUG-14 đáng đọc nhất, và nó là lý do bộ test này có giá trị hơn tổng các phần của nó. Ba lỗi riêng lẻ — thiếu auth (Critical nhưng "chỉ" sửa dữ liệu), partial update ghi NULL (High), `price.toString()` theo chuỗi/lẻ id (Medium, trông như lỗi làm màu) — khi ghép lại cho phép **một người không có tài khoản làm sập toàn bộ hệ thống bằng 2 request**, lặp lại được sau mỗi lần restart. Không có test case đơn lẻ nào bắt được nó; nó lộ ra khi chạy chuỗi request thật và **SUT chết giữa lượt probe**.

4 giả thuyết đã bị loại sau khi kiểm chứng (cách ly giỏ, SEC-02 ở `POST /api/cart`, `DROP TABLE` thực sự xóa bảng, SQLi ở `:id`) — ghi lại trong bug report để không nhận vơ.

8. Postman feature đã dùng (§6)

#	Feature	Dùng ở đâu
1	Collection + folder lồng nhau	3 collection · 27 folder (<code>00-setup</code> → <code>99-teardown</code>)
2	Environment + biến	<code>HW06-local-23127178</code> , 17 biến
3	Biến secret	<code>admin_password</code> , <code>user_password</code> , <code>user2_password</code>

#	Feature	Dùng ở đâu
4	Pre-request script cấp collection	header <code>X-Student-Id</code> cho mọi request (§6.4)
5	Test script <code>pm.test</code>	372 assertion (+253 ở regression suite)
6	JSON schema validation	<code>pm.response.to.have.jsonSchema</code> — folder <code>40-schema</code> của cả 3 API
7	Biến động giữa request	<code>pm.environment.set</code> cho token, <code>product_id</code> , <code>total_products</code> , <code>cart_before</code>
8	<code>pm.sendRequest</code>	dọn fixture ở <code>00-setup</code> và <code>99-teardown</code> (giữ mỗi lượt độc lập)
9	<code>pm.iterationData</code> + file CSV	<code>postman/data/*.csv</code> cho Collection Runner (data-driven)
10	<code>console.log</code> → Postman Console	bảng chứng §11 cho header sinh viên
11	Newman CLI + reporter htmlextra	<code>tools/run-newman.sh</code>
12	Newman JSON reporter	nguồn của <code>summary.md</code> và của cổng CI
13	Newman trong CI/CD (GitHub Actions)	<code>.github/workflows/api-tests.yml</code>
14	Collection description / documentation	mỗi collection ghi rõ được sinh từ file spec nào

Chưa dùng: **Mock server** và **Monitor** — cả hai cần Postman Cloud; bài này chạy hoàn toàn local + CI, và `tools/gen-artifacts.mjs` đóng vai trò "nguồn spec" mà mock server sẽ đảm nhiệm. Ghi rõ thay vì đánh dấu cho đủ danh sách.

9. CI/CD (§6)

Cấu hình và hai lượt mẫu: `ci/ci-report.md`. Pipeline: `.github/workflows/api-tests.yml`.

Hai bộ test, hai cổng, hai vai trò. Đề đòi một lượt *tất cả test case pass* và một lượt *có một test fail*. Bộ test chính không đáp ứng được theo nghĩa chữ, nên pipeline có thêm **regression suite** — tập con các case đang xanh, **sinh tự động** bởi `tools/gen-regression.mjs`, giữ nguyên expected, cổng **0 đỏ**. Hai lượt mẫu: **XANH 216/0** · **ĐỎ 1/217**.

Vì sao bộ bug-hunting vẫn dùng cổng baseline chứ không phải 0. Bộ test cố ý bắt bug thật nên số assertion đỏ ở trạng thái bình thường là **89**, không phải 0. Lấy "0 đỏ" làm cổng thì pipeline đỏ vĩnh viễn và **hồi quy mới không còn phân biệt được với bug cũ** — cổng mất hết tác dụng. `tools/ci-gate.mjs` so với `ci/expected-failures.json` (29 / 30 / 30, mỗi mục kèm lý do trở tới bug): đỏ **tăng** = hồi quy mới, đỏ **giảm** = SUT đã sửa hoặc test của mình yếu đi, cả hai đều cần người xem lại. Chế độ `gate_mode=strict` (đỏ nếu có bất kỳ assertion đỏ) là cách tạo **lượt ĐỎ mẫu** mà §6 đòi, không cần cố tình viết một test sai vào repo.

10. AI test generator (§7)

Thiết kế, sơ đồ và pseudocode: `generator/design.md`.

Điểm khác biệt so với một bản thiết kế trên giấy: generator này **đã chạy và sinh ra chính bộ test của bài này**. `tools/gen-artifacts.mjs` đọc `generator/specs/<api>.mjs` rồi sinh: `generated.md` (case AI), `audit.md` (case AI + nhãn audit), `extended.md` (case sinh viên + bảng "vì sao AI bỏ sót"), và collection Postman — 157 case, 372 assertion, từ **một** nguồn định nghĩa.

11. Human review — AI sai và bỏ sót gì

Đây là mục §2 và §10 chấm nặng nhất. Bảng dưới là **11 lỗi thật đã bắt được**, mỗi dòng ghi **cách phát hiện** — vì "đã review, không thấy lỗi" là câu trả lời tệ nhất có thể.

#	Lỗi của AI	Loại	Phát hiện bằng cách nào	Đã sửa thế nào
1	TC-PRODLIST-011: expected 0 dòng cho <code>search=""</code> — bịa vì spec không nói gì về trim	thiết kế test	Soát cột <code>Căn cứ</code> : không trỏ được vào mục nào của spec	Hạ về đúng phần spec bảo đảm: 200 + schema. Nếu để nguyên sẽ sinh ra bug giả
2	TC-CART-008: expected "từ chối vì vượt tồn kho" — bảng <code>products</code> không có cột tồn kho	thiết kế test	Đối chiếu <code>database.js:64–72</code>	Giữ case (yêu cầu FR-07 vẫn tồn tại) + ghi rõ hạn chế mô hình dữ liệu
3	TC-PRODUPD-005: ghi cứng 400 cho <code>name</code> 300 ký tự — spec không nêu giới hạn	thiết kế test	Soát <code>Căn cứ</code>	Hạ về "không 500 + vẫn là JSON"
4	Bỏ sót hoàn toàn phân vùng tiếng Việt chữ thường có dấu	bỏ sót (đặc điểm API)	Tự đọc dữ liệu fixture và thử <code>áo</code> bằng curl	Thêm TC-101 → BUG-05
5	Coi <code>%</code> chỉ là payload SQLi, bỏ mất <code>%</code> trong dữ liệu hợp lệ ("bàn phím 100%")	bỏ sót (model)	Nhìn danh sách fixture, thấy <code>%</code> là ký tự bình thường trong tên	Thêm TC-102/103 → BUG-06
6	Test security dừng ở status code, không kiểm hệ quả	bỏ sót (model)	Nhận ra SUT trả 200 cho mọi thứ → status code không phân biệt "đã validate" với "nhận bừa"	Thêm 6 case verify (TC-PRODLIST-106, TC-CART-102/104, TC-PRODUPD-101/103/106) → nâng BUG-13 lên Critical
7	Không kiểm response lỗi (content-type, rò rỉ thông tin)	bỏ sót (prompt)	Thấy <code>'</code> trả 500 trong lúc probe	Thêm TC-105 → BUG-02
8	Chỉ kiểm endpoint được giao, không soát route lân cận cùng nhóm quyền	bỏ sót (prompt)	Đọc <code>server.js</code> quanh dòng 179 thấy <code>POST / DELETE</code> cũng thiếu middleware	Thêm TC-PRODUPD-107/108 → mở rộng BUG-13
9	Sinh mã JS có lỗi cú pháp : giá trị chuỗi lồng trong tên <code>pm.test</code> không escape dấu <code>"</code>	kỹ thuật	Lướt Newman đầu: 2 case đỏ với <code>missing) after argument list</code> — không phải bug SUT	Sửa hàm <code>fieldEq</code> trong generator (escape <code>"</code> → <code>'</code>) rồi sinh lại

#	Lỗi của AI	Loại	Phát hiện bằng cách nào	Đã sửa thế nào
10	Chạy seed trước khi SUT seed xong DB → user2 bị xoá cùng bảng <code>users</code>	kỹ thuật	Lướt đầu: SETUP-03 của API-02 đồ 401 — đồ vì môi trường, không vì bug	Điều kiện "SUT sẵn sàng" đổi thành <i>login admin được</i> , không chỉ <i>cổng đã mở</i>
11	TC-CART-020 và TC-CART-021: assertion nghiêm hơn expected của chính nó — cột expected ghi " <i>từ chối, hoặc lấy giá/tên từ catalog</i> " nhưng assertion chỉ nhận <code>400/422</code> . Nếu SUT chọn hành vi thứ hai thì test đồ oan → sẽ bị báo thành bug của SUT	thiết kế test	soát chéo cột <code>expect</code> với <code>checks</code> khi tự chấm lại bài	Không nói assertion cho qua: chuyển phần khẳng định sang TC-CART-107 , nơi cả hai hành vi hợp lệ đều dẫn tới cùng một kết luận kiểm được (giả không được chứa dòng thiếu giá / không tên). Bug vẫn bị bắt, bằng case không tranh cãi được về expected
12	Cổng baseline quét luôn <code>ci-regression.json</code> → build đồ vì " <i>chưa có baseline</i> " trong khi cả hai cổng thật đều xanh	kỹ thuật	đọc log lướt CI đầu sau khi thêm regression suite	thu hẹp glob về <code>ci-api-*.json</code> ; regression có cổng riêng
13	Bản sửa cho lỗi #10 vẫn chưa đủ: lấy "login admin được" làm mốc SUT sẵn sàng — nhưng mốc đó cũng sai	kỹ thuật	Chạy lại toàn bộ để kiểm tái lập: API-02 ra 34 thay vì 30. Tái hiện thủ công 3/3 lần : <code>POST /api/register</code> trả <code>200 {"id":3}</code> rồi user biến mất	Mốc sẵn sàng đổi thành ghi rồi kiểm chứng bản ghi còn sống (thử tối đa 12 lần); seed thất bại thì chặn lượt chạy thay vì cảnh báo
14	Xuất Excel đếm đúng case AI (250 thay vì 136)	prompt quality	Số không khớp <code>summary.md</code>	Bỏ <code>generated.md</code> khỏi sheet gộp khi đã có <code>audit.md</code>
15	<code>verify-all.sh</code> đếm dòng thay vì TC ID duy nhất → mỗi API phỏng thêm bằng số dòng bằng "vì sao AI bỏ sót" (50 thay vì 43)	kỹ thuật	Số không khớp <code>summary.md</code> và <code>excel/</code>	Đếm <code>sort -u</code> trên TC ID
16	Script giữ pipe của tiến trình con → lệnh gọi treo tới timeout	model limitations	<code>verify-bugs.sh</code> bị timeout 2 phút	Thêm <code>< /dev/null</code> khi khởi động SUT
17	Ghi 2 file vào repo HW05 do shell giữ cwd giữa các lệnh	quy trình	Đối chiếu <code>git status</code> của HW05	Chuyển file về HW06, hoàn nguyên HW05, ghi vào AI audit
18	README + main-report còn ghi 329 assertion (số thật 333) ở 3 chỗ	kỹ thuật	<code>tools/check-claims.mjs</code> soát mọi con số công bố so với raw JSON	sửa 3 chỗ; phép kiểm vào <code>npm run verify</code> mục 5b
19	<code>ci-report.md</code> ghi 2 hash commit KHÔNG TỒN TẠI — biết 1 hash thật rồi diễn nốt 2 hash theo trí nhớ	model limitations	soát lần hai: <code>git log --format=%s -1 <hash></code> trả rỗng	tra <code>git log</code> lấy hash thật; <code>check-claims.mjs</code> giờ kiểm hash bằng <code>git cat-file</code>

#	Lỗi của AI	Loại	Phát hiện bằng cách nào	Đã sửa thế nào
20	Bảng §11 đánh số trùng, sai thứ tự, thiếu 1 dòng so với bảng trong AI audit	kỹ thuật	đếm lại dãy số	đánh số lại liên tục, thêm dòng thiếu, sửa 3 tham chiếu chéo
21	mục <i>Giới hạn</i> (bản trước) trở tới file lượt chạy đã bị thay	kỹ thuật	so tên file với <code>ls reports/newman/</code>	cập nhật theo lượt sinh viên tự chạy
22	TC-CART-101/102 dán nhãn Security nhưng không trở được SEC-0x nào — dùng "security" theo nghĩa thông thường thay vì theo SEC-01...07	prompt quality	<code>tools/check-cases.mjs</code> — bắt biến "case Security phải trở một SEC-0x"	ghi rõ "ngoài SEC-01...07" + ghi lỗi hỏng của danh sách SEC vào traceability
23	Dán nhãn SV cho 22 case do AI sinh và gọi là "sinh viên tự thêm" — misattribution, không thoả §6.3 "of your own"	quy trình	soát lại lần bốn, đối chiếu <code>extended.md</code> với chính lời khai ở Interaction #5 ("AI output ... thêm 22 case")	đổi nhãn thành AI-2, viết cảnh báo vào đầu <code>extended.md</code> , tạo <code>own.md</code> cho case của sinh viên, và thêm bất biến chặn nhãn SV
24	<code>check-submission.mjs</code> coi mọi lỗi gh là "issue KHÔNG tồn tại" → chạy trong shell chưa <code>gh auth login</code> là ra 3 dòng đỏ khẳng định sai, và người chấm trừ điểm phần bug report	kỹ thuật	issue #323/#328/#341 bị báo không tồn tại, kiểm lại bằng API công khai thì đều OPEN	gọi API công khai bằng <code>curl</code> , chỉ HTTP 404 là "không tồn tại"; 000/403 là "không kiểm được"
25	Kết luận "bug" khi expected không có căn cứ bắt buộc — 3/6 phát hiện từ case sinh viên chọn bị báo thành bug: thiếu phân trang (spec §3.1 không định nghĩa <code>limit / page</code>), giữ giá cũ (price-snapshot là chính sách hợp lệ, spec im lặng), <code><script></code> lưu nguyên văn (SEC-04 nói escape khi hiển thị , không cấm lưu)	thiết kế test	bản review đối chiếu từng expected với câu chữ spec/SEC	Hạ xuống R-01, Q-01, R-02 ; đổi 3 case thành <i>characterization test</i> ; sửa nhãn 3 issue công khai (#402 → RISK, #403 → QUESTION, #404 → RISK) kèm comment giải thích
26	Hai lượt CI không đúng nghĩa §6 ("all test cases passing" / "one test case failing") — bộ test chính luôn có assertion đỏ (lúc đó 89) nên lượt "xanh" vẫn đầy assertion đỏ	thiết kế CI	Đọc lại đúng câu chữ §6 khi tự chấm bài theo bảng §15	Thêm regression suite sinh tự động (216 assertion, 0 đỏ) với cổng riêng; lượt đỏ tạo bằng đúng 1 assertion có nhãn DEMO rồi gỡ ngay ở commit sau

Ba lỗi đáng giá nhất về mặt phương pháp — #9 (mã JS sinh sai), #10 (mốc sẵn sàng sai) và #13 (bản sửa cho #10 vẫn sai): cả ba đều làm test case đỏ, và nếu không truy nguyên thì sẽ được **báo thành bug của SUT**. Riêng chuỗi #10 → #13 đáng đọc kỹ, vì nó là một **bản sửa sai được sửa lại**:

- Lần đầu: user2 mất → chẩn đoán "SUT chưa seed xong" → sửa mốc sẵn sàng thành *login admin được*. Lượt chạy sau đó xanh 29/30/30, và **tôi đã tưởng là xong**.
- Lượt kiểm tái lập: API-02 quay lại 34 đỏ. Tái hiện thủ công 3/3 lần cho thấy `POST /api/register` trả `200 {"id":3}` rồi user biến mất — nghĩa là SUT **phục vụ request bằng dữ liệu cũ còn trong database.sqlite** trong lúc `DROP TABLE users` chưa chạy. Dòng log `Database initialized and seeded` in ra **trước** khi SQL chạy xong (thấy rõ trong `.run-logs/sut.log`: nó là dòng **đầu tiên**, trước cả `Server is running`). Nên "login admin được" cũng là mốc sai — nó chỉ chứng minh có *một bảng users* nào đó có *dữ liệu*, không chứng minh bằng đó là bảng mới.

- Bản sửa đúng: mốc sẵn sàng = **ghi một bản ghi rồi đọc lại thấy nó vẫn còn**; và seed thất bại thì **chặn** lượt chạy thay vì in cảnh báo rồi chạy tiếp.

Bài học rút ra không phải "kiểm tra môi trường trước khi chạy" — mà là: **một bản sửa chỉ được coi là đúng khi lượt chạy tái lập được**. Nếu bài này dừng ở lần sửa đầu, bộ nộp sẽ có 4 assertion đỏ ngẫu nhiên và một câu kết luận sai về nguyên nhân.

Mỗi assertion đỏ phải trả lời được: *đỏ vì SUT sai, vì test tôi viết sai, hay vì môi trường?*

Năm lượt soát sau khi bài đã "xong" — và số lỗi tìm được chưa lần nào về 0:

Lượt t	Soát gì	Tìm được
1	chạy lại toàn bộ, kiểm tái lập	1 (lỗi #10 → #13: mốc sẵn sàng vẫn sai)
2	đọc tài liệu với vai người chấm	3 (#19 hash bịa · #20 bảng đánh số trùng · #21 trỏ file cũ)
3	soát nội dung 114 nhãn VALID	2 (#22 nhãn Security không trỏ được SEC · #18 số 329 còn sót)
4	toàn vẹn bộ nộp (bug→case, issue, ảnh, PDF, hash)	4 (#19 hash bịa · #23 misattribution nhãn SV · #24 checker báo sai issue · placeholder)
5	đối chiếu từng expected với câu chữ spec/SEC	1 lỗi lớn (#25: 3 phát hiện bị kết luận là bug khi không có căn cứ bắt buộc)

Mỗi lượt sinh ra một phép kiểm bằng máy để lỗi đó không quay lại âm thầm: [check-expect-vs-checks.mjs](#) (135 case · 0 lệch) · [check-claims.mjs](#) (18 khớp · 0 lệch — gồm kiểm hash commit bằng git cat-file) · [check-cases.mjs](#) (157 case · 8 bất biến nội dung). Cả bốn nằm trong `npm run verify` (**37 PASS · 0 FAIL**).

Lượt soát lần hai (23/08/2026): đọc lại toàn bộ với vai người chấm và tìm thêm **3 lỗi tài liệu** mà không lượt nào trước đó thấy: (a) `ci/ci-report.md` ghi **2 hash commit không tồn tại** cho lượt CI đỏ — loại lỗi tệ nhất trong một báo cáo vì nó là chi tiết *kiểm được ngay* và sai; (b) bảng §11 bị **đánh số trùng** (...10, 16, 17, 12, 13...) và **thiếu một dòng** so với bảng trong AI audit; (c) mục *Giới hạn* (bản trước) vẫn trỏ tới file lượt chạy **đã bị thay**. Đã sửa cả ba.

Từ đó rút ra một phép kiểm mới, biến lỗi #11 thành **bất biến kiểm được bằng máy** thay vì trông vào mắt người: [tools/check-expect-vs-checks.mjs](#) rút tập status code nêu ở cột `status` của **mọi** case rồi so với tập status mà `checks` thực sự chấp nhận. Kết quả hiện tại: **135 case có assert status · 0 case lệch**. Phép kiểm này đã vào `npm run verify`.

Cách làm để trả lời được câu đó: viết `bug-report/verify-bugs.sh` — tái hiện từng bug bằng `curl` độc lập với Postman. 19/19 bug tái hiện được; 4 giả thuyết **không** tái hiện được đã bị loại khỏi báo cáo.